CHAPTER

15 QUALITY CONCEPTS

The drumbeat for improved software quality began in earnest as software became increasingly integrated in every facet of our lives. By the 1990s, major corporations recognized that billions of dollars each year were being wasted on software that didn't deliver the features and functionality that were promised. Worse, both government and industry became increasingly concerned that a major software fault might cripple important infrastructure, costing tens of billions more. By the turn of the century, *CIO Magazine* trumpeted the headline, "Let's Stop Wasting $78 Billion a Year," lamenting the fact that "American businesses spend billions for software that doesn't do what it's supposed to do" [Lev01]. Sadly, at least one survey of the state of software quality practices done in 2014 suggests that maintenance and software evolution activities make up as much as 90 percent of the total software development costs [Nan14]. Poor software quality caused by a rush to release products without adequate testing continues to plague the software industry.

KEY CONCEPTS

cost of quality 317
good enough 316
liability 320
machine learning. 322
management actions 321
quality 311
quality dilemma. 315
quality dimensions 314
quality factors 312
quantitative quality assessment. 315
risks 319
security 320

QUICK LOOK

What is it? The answer isn't as easy as you might think. You know quality when you see it, and yet, it can be an elusive thing to define. But for computer software, quality is something that we must define, and that's what we'll do in this chapter.

Who does it? Everyone—software engineers, managers, all stakeholders—involved in the software process is responsible for quality.

Why is it important? You can do it right, or you can do it over again. If a software team stresses quality in all software engineering activities, it reduces the amount of rework that it must do. That results in lower costs, and more importantly, improved time to market.

What are the steps? To achieve high-quality software, four activities must occur: proven software engineering process and practice, solid project management, comprehensive quality control, and the presence of a quality assurance infrastructure.

What is the work product? Software that meets its customer's needs, performs accurately and reliably, and provides value to all who use it.

How do I ensure that I've done it right? Track quality by examining the results of all quality control activities, and measure quality by examining errors before delivery and defects released to the field.

Today, software quality remains an issue, but who is to blame? Customers blame developers, arguing that sloppy practices lead to low-quality software. Developers blame customers (and other stakeholders), arguing that irrational delivery dates and a continuing stream of changes force them to deliver software before it has been fully validated. Who's right? *Both*—and that's the problem. In this chapter, we consider software quality as a concept and examine why it's worthy of serious consideration whenever software engineering practices are applied.

15.1 What Is Quality?

In his mystical book *Zen and the Art of Motorcycle Maintenance,* Robert Persig [Per74] commented on the thing we call *quality*:

> Quality . . . you know what it is, yet you don't know what it is. But that's self-contradictory. But some things are better than others; that is, they have more quality. But when you try to say what the quality is, apart from the things that have it, it all goes poof! There's nothing to talk about . . . Obviously some things are better than others . . . but what's the betterness? . . . So round and round you go, spinning mental wheels and nowhere finding anyplace to get traction. What the hell is Quality? What is it?

Indeed—what is it?

At a somewhat more pragmatic level, David Garvin [Gar84] of the Harvard Business School suggests that "quality is a complex and multifaceted concept" that can be described from five different points of view. The *transcendental view* argues (like Persig) that quality is something you immediately recognize but cannot explicitly define. The *user view* sees quality in terms of an end user's specific goals. If a product meets those goals, it exhibits quality. The *manufacturer's view* defines quality in terms of the original specification of the product. If the product conforms to the spec, it exhibits quality. The *product view* suggests that quality can be tied to inherent characteristics (e.g., functions and features) of a product. Finally, the *value-based view* measures quality based on how much a customer is willing to pay for a product. In reality, quality encompasses all these views and more.

Quality of design refers to the characteristics that designers specify for a product. The grade of materials, tolerances, and performance specifications all contribute to the quality of design. As higher-grade materials are used, tighter tolerances and greater levels of performance are specified, and the design quality of a product increases if the product is manufactured according to specifications.

In software development, quality of design encompasses the degree to which the design meets the functions and features specified in the requirements model. *Quality of conformance* focuses on the degree to which the implementation follows the design and the resulting system meets its requirements and performance goals.

But are quality of design and quality of conformance the only issues that software engineers must consider? Robert Glass [Gla98] argues that a more "intuitive" relationship is in order:

$$\text{user satisfaction} = \text{compliant product} + \text{good quality} + \text{delivery within budget and schedule}$$

At the bottom line, Glass contends that quality is important, but if the user isn't satisfied, nothing else really matters. DeMarco [DeM98] reinforces this view when he states: "A product's quality is a function of how much it changes the world for the better." This view of quality contends that if a software product provides substantial benefit to its end users, they may be willing to tolerate occasional reliability or performance problems. A modern view of software quality requires attention to customer satisfaction as well as conformance to the product requirements [Max16].

15.2 SOFTWARE QUALITY

Even the most jaded software developers will agree that high-quality software is an important goal. But how do we define *software* quality? In the most general sense, software quality can be defined as: *An effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it.*

There is little question that the preceding definition could be modified or extended and debated endlessly. For the purposes of this book, the definition serves to emphasize three important points:

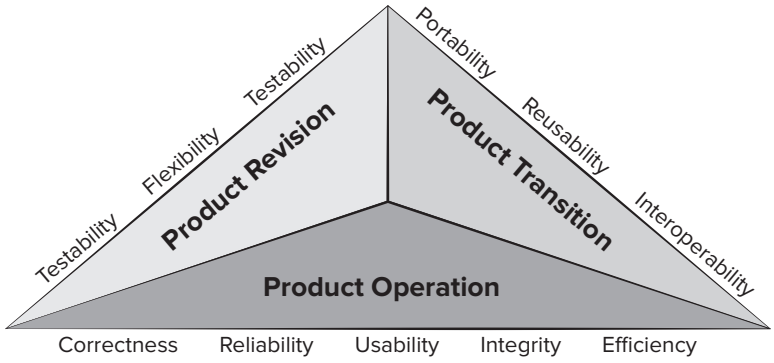
1. An *effective software process* establishes the infrastructure that supports any effort at building a high-quality software product. The management aspects of process create the checks and balances that help avoid project chaos—a key contributor to poor quality. Software engineering practices allow the developer to analyze the problem and design a solid solution—both critical to building high-quality software. Finally, umbrella activities such as change management and technical reviews have as much to do with quality as any other part of software engineering practice.
2. A *useful product* delivers the content, functions, and features that the end user desires, but as important, it delivers these assets in a reliable, error-free way. A useful product always satisfies those requirements that have been explicitly stated by stakeholders. In addition, it satisfies a set of implicit requirements (e.g., ease of use) that are expected of all high-quality software.
3. By *adding value for both the producer and user* of a software product, high-quality software provides benefits for the software organization and the end-user community. The software organization gains added value because high-quality software requires less maintenance effort, fewer bug fixes, and reduced customer support. This enables software engineers to spend more time creating new applications and less on rework. The user community gains added value because the application provides a useful capability in a way that expedites some business process. The end result is (1) greater software product revenue, (2) better profitability when an application supports a business process, and/or (3) improved availability of information that is crucial for the business.

15.2.1 Quality Factors

Several software quality models and standards have been proposed in the software engineering literature. David Garvin [Gar84] writes that quality is a multifaceted

FIGURE 15.1

McCall's software quality factors



phenomena and requires the use of multiple perspectives to assess it. McCall and Walters [McC77] proposed a useful way to think about and organize factors affecting software quality. Their software quality factors (shown in Figure 15.1) focus on three software product aspects: its operation characteristics, its ability to undergo change, and its adaptability to new environments. McCall's quality factors provide a basis for engineering software that provides high levels of user satisfaction by focusing on the overall user experience delivered by the software product. This cannot be done unless developers ensure that the requirements specification is correct and that defects are removed early in the software development process [Max16].

The ISO 25010 quality model is the newest standard (created in 2011 and revised in 2017).[1] This standard defines two quality models. The *quality in use* model describes five characteristics that are appropriate when considering using the product in a particular context (e.g., using the product on a specific platform by a human). The *product quality* model describes eight characteristics that focus on both the static and dynamic nature of computer systems.

- **Quality in Use Model**
  - **Effectiveness.** Accuracy and completeness with which users achieve goals
  - **Efficiency.** Resources expended to achieve user goals completely with desired accuracy
  - **Satisfaction.** Usefulness, trust, pleasure, comfort
  - **Freedom from risk.** Mitigation of economic, health, safety, and environmental risks
  - **Context coverage.** Completeness, flexibility
- **Product Quality**
  - **Functional suitability.** Complete, correct, appropriate
  - **Performance efficiency.** Timing, resource utilization, capacity
  - **Compatibility.** Coexistence, interoperability

1 The ISO 25010 can be found at https://www.iso.org/standard/35733.html.

- **Usability.** Appropriateness, learnability, operability, error protection, aesthetics, accessibility
- **Reliability.** Maturity, availability, fault tolerance, recoverability
- **Security.** Confidentiality, integrity, accountability, authenticity
- **Maintainability.** Modularity, reusability, modifiability, testability
- **Portability.** Adaptability, installability, replaceability

The addition of the quality in use model helps to emphasize the importance of customer satisfaction in the assessment of software quality. The product quality model points out the importance of assessing both the functional and nonfunctional requirements for the software product [Max16].

15.2.2 Qualitative Quality Assessment

The quality dimensions and factors presented in Section 15.2.1 focus on the complete software product and can be used as a generic indication of the quality of an application. Your software team can develop a set of quality characteristics and associated questions that would probe the degree to which each factor has been satisfied.[2] For example, ISO 25010 identifies *usability* as an important quality factor. If you were asked to evaluate a user interface and assess its usability, how would you proceed?

Although it's tempting to develop quantitative measures for the quality factors noted in Section 15.2.1, you can also create a simple checklist of attributes that provide a solid indication that the factor is present. You might start with the sub-characteristics suggested for usability in ISO 25010: appropriateness, learnability, operability, error protection, aesthetics, and accessibility. You and your team might decide to create a user questionnaire and a set of structured tasks for users to perform. You might observe the users while they perform these tasks and have them complete the questionnaire when they finish. We will discuss usability testing in more detail in Chapter 21.

To conduct your assessment, you and your team will need to address specific, measurable (or at least, recognizable) attributes of the interface. Your tasks might be focused on answering the following questions:

- How quickly can users determine whether the software product can be used to help them complete their task or not? (appropriateness)
- How long does it take users to learn how to use the system functions needed to complete their task? (learnability)
- Is the user able to recall how to use system functions in subsequent testing sessions without having to relearn them? (learnability)
- How long does it take users to complete tasks using the system? (operability)
- Does the system try to prevent users from making errors? (error protection)
- Does the system allow users to undo operations that may have resulted in errors? (error protection)
- Do answers give favorable responses to questions about the appearance of the user interface? (aesthetics)

2 These characteristics and questions would be addressed as part of a software review (Chapter 16).

- Does the interface conform to the expectations set forth by the golden rules from Chapter 12? (accessibility)
- Does the user interface conform to the accessibility checklist items required for the intended users? (accessibility)

As the interface design is developed, the software team would review the design prototype and ask the questions noted. If the answer to most of these questions is yes, it is likely that the user interface exhibits high quality. A collection of questions similar to these would be developed for each quality factor to be assessed. In the case of usability, it is always important to observe representative users interact with the system. For some other quality factors it may be important to test the software in the wild (or at least in the production environment).

15.2.3 Quantitative Quality Assessment

In the preceding subsections, we have presented a set of qualitative factors for the "measurement" of software quality. The software engineering community strives to develop precise measures for software quality and is sometimes frustrated by the subjective nature of the activity. Cavano and McCall [Cav78] discuss this situation:

> Subjectivity and specialization . . . apply to determining software quality. To help solve this problem, a more precise definition of software quality is needed as well as some way to derive quantitative measurements of software quality for objective analysis . . . Since there is no such thing as absolute knowledge, one should not expect to measure software quality exactly, for every measurement is partially imperfect.

Several software design defects can be detected using software metrics. The process consists of finding code fragments that suggest the presence of things like high coupling or unnecessary levels of complexity. Internal code attributes can be described quantitatively using software metrics. Any time software metric values computed for a code fragment fall outside the range of acceptable values, it signals the existence of a quality problem that should be investigated [Max16].

In Chapter 23, we'll present a set of software metrics that can be applied to the quantitative assessment of software quality. In all cases, the metrics represent indirect measures; that is, we never really measure *quality* but rather some manifestation of quality. The complicating factor is the precise relationship between the variable that is measured and the quality of software.

15.3 The Software Quality Dilemma

In an interview [Ven03] published on the Web, Bertrand Meyer discusses what we call the *quality dilemma:*

> If you produce a software system that has terrible quality, you lose because no one will want to buy it. If on the other hand you spend infinite time, extremely large effort, and huge sums of money to build the absolutely perfect piece of software, then it's going to take so long to complete and it will be so expensive to produce that you'll be out of business anyway. Either you missed the market window, or you simply exhausted all your resources. So people in industry try to get to that magical middle ground where the product is good enough not to be rejected right away, such as during evaluation, but also not the object of so much perfectionism and so much work that it would take too long or cost too much to complete.